

Q1. E-Commerce Product Records – File Organization Strategy

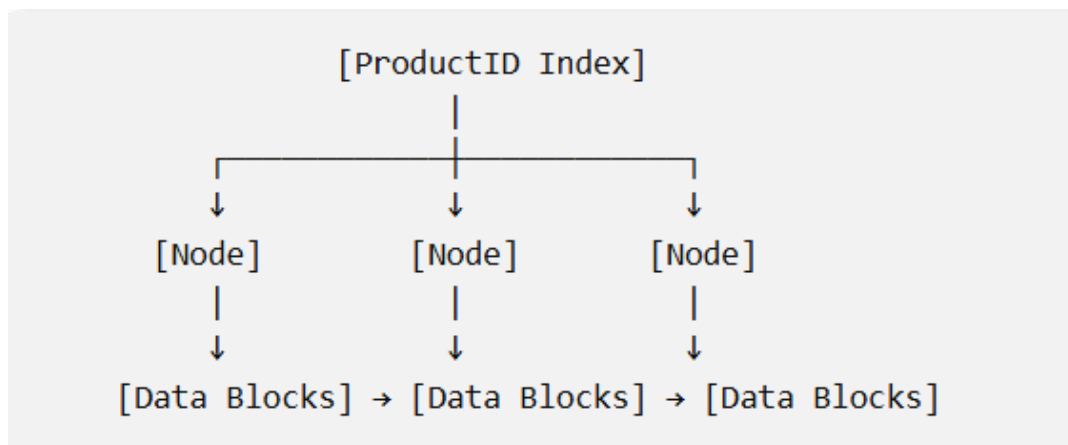
Answer

For an e-commerce company storing **10 million product records**, the recommended file organization is a **B+ Tree-based indexed file organization** (with records stored in pages/blocks and a B+ Tree index on ProductID).

1. Recommended Strategy: B+ Tree Indexed File

A B+ Tree provides efficient searching while also supporting insertion, deletion, and range queries.

Structure:



Each leaf node contains pointers to the actual product records. Leaf nodes are linked, making sequential and range retrieval efficient.

2. Why B+ Tree is Suitable

Requirement	B+ Tree
Search product by ID	Very fast
Insert new product	Efficient
Delete product	Efficient
Range search	Very efficient
Sequential access	Efficient
Large dataset	Highly suitable

Scalability High

For an e-commerce system, customers may search products by **ProductID**, price range, category, or other indexed attributes. B+ Trees are particularly useful when both point and range searches are required.

3. Cost Analysis

Assume:

- Number of records = **10,000,000**
- Average record size = **200 bytes** (*assumption for analysis*)

- Storage required:

$$10,000,000 \times 200 = 2,000,000,000 \text{ bytes}$$

Approximately:

$$\frac{2,000,000,000}{1024^3} \approx 1.86 \text{ GB}$$

If a **4 KB block** is used:

$$\frac{2,000,000,000}{4096} \approx 488,282 \text{ blocks}$$

A B+ Tree does not require scanning all **488,282 blocks** for every search. Instead, it follows the index from the root to the appropriate leaf, resulting in approximately **O(log N)** search complexity.

4. Cost Comparison

File Organization	Search	Insert	Delete	Range Query	Suitability
Heap File	O(N)	Very fast	O(N)	Poor	Low
Sorted File	O(log N)	Costly	Costly	Very good	Medium
B+ Tree Indexed	O(log N)	Efficient	Efficient	Excellent	High

5. Conclusion

For **10 million e-commerce product records**, a **B+ Tree indexed file organization** is the most suitable choice. Although maintaining the index introduces additional storage and

update costs, the significant reduction in search and range-query costs makes it appropriate for a large-scale e-commerce database.

Final Recommendation:

Use **B+ Tree indexing with block-based data storage**, particularly on frequently searched attributes such as ProductID, Category, and Price.

Q2. Banking System – Indexing Strategy

Answer

For a banking transaction system, a **B+ Tree indexing strategy** is suitable because the system needs both:

1. Fast retrieval using **Account Number**
2. Efficient **range queries using Transaction Date**

1. Transaction Table

Assume the transaction table contains:

Attribute	Description
Transaction_ID	Unique transaction identifier
Account_No	Customer account number
Transaction_Date	Date of transaction
Amount	Transaction amount
Transaction_Type	Deposit/Withdrawal

2. Indexing Strategy

Index 1: Account Number

Create a **B+ Tree index** on Account_No.

```
CREATE INDEX idx_account
```

```
ON Transactions(Account_No);
```

This allows the database to quickly locate all transactions belonging to a particular account.

Example query:

```
SELECT *  
  
FROM Transactions  
  
WHERE Account_No = 10025;
```

Instead of scanning the entire transaction table, the index can directly locate the required records.

Index 2: Date

Create a **B+ Tree index** on Transaction_Date.

```
CREATE INDEX idx_transaction_date  
  
ON Transactions(Transaction_Date);
```

This is useful for date-range queries.

Example:

```
SELECT *  
  
FROM Transactions  
  
WHERE Transaction_Date  
  
BETWEEN '2026-01-01' AND '2026-01-31';
```

A B+ Tree is particularly suitable for range searches because its leaf nodes are linked sequentially.

3. Composite Index

For queries that commonly retrieve transactions for a **specific account within a date range**, a composite B+ Tree index is even more effective:

```
CREATE INDEX idx_account_date  
  
ON Transactions(Account_No, Transaction_Date);
```

For example:

```
SELECT *
```

FROM Transactions

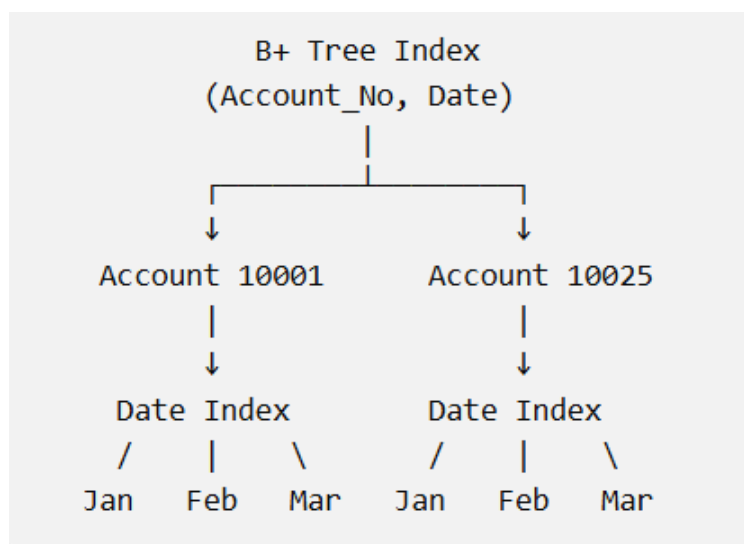
WHERE Account_No = 10025

AND Transaction_Date

BETWEEN '2026-01-01' AND '2026-01-31';

The index first narrows the records using Account_No and then efficiently searches the specified date range.

4. Proposed Index Structure



5. Why B+ Tree?

Requirement	B+ Tree
Search by Account No.	Excellent
Date-range search	Excellent
Combined Account + Date search	Excellent
Sequential date access	Efficient
Large transaction table	Suitable
Insert/Delete	Efficient

6. Cost Analysis

Without an index, the database may need to perform a **full table scan**, examining a large number of transaction records.

With the B+ Tree index, the search generally requires approximately:

[
 $O(\log N)$
]

index traversal operations, followed by retrieval of the matching records.

The composite index also reduces unnecessary data-page accesses because the search is narrowed first by Account_No and then by Transaction_Date.

Conclusion

The recommended strategy is to use a **B+ Tree composite index on (Account_No, Transaction_Date)**. If independent searches by account number and date are also frequent, separate B+ Tree indexes can additionally be maintained. This provides fast point queries, efficient date-range retrieval, and good performance for large banking transaction databases.

Q3. Healthcare System – Multi-Level Indexing

Answer

For a healthcare database, patient records need to be retrieved using two different attributes:

1. **PatientID** – for direct identification of a patient.
2. **Doctor Name** – to retrieve all patients associated with a particular doctor.

A **multi-level indexing approach** can be designed using **B+ Tree indexes**.

1. Patient Table

Assume the table contains:

Attribute	Description
PatientID	Unique patient identifier
PatientName	Name of patient
DoctorName	Assigned doctor

Age	Patient age
Diagnosis	Diagnosis details

2. Primary Index – PatientID

Create a B+ Tree index on PatientID.

```
CREATE INDEX idx_patient_id
```

```
ON Patient(PatientID);
```

Since PatientID is unique, this index provides fast direct access to an individual patient's record.

Example:

```
SELECT *
```

```
FROM Patient
```

```
WHERE PatientID = 10525;
```

The index navigates through multiple levels until it reaches the leaf containing the required PatientID.

3. Secondary Index – Doctor Name

Create a secondary B+ Tree index on DoctorName.

```
CREATE INDEX idx_doctor_name
```

```
ON Patient(DoctorName);
```

This allows the system to efficiently retrieve all patients assigned to a particular doctor.

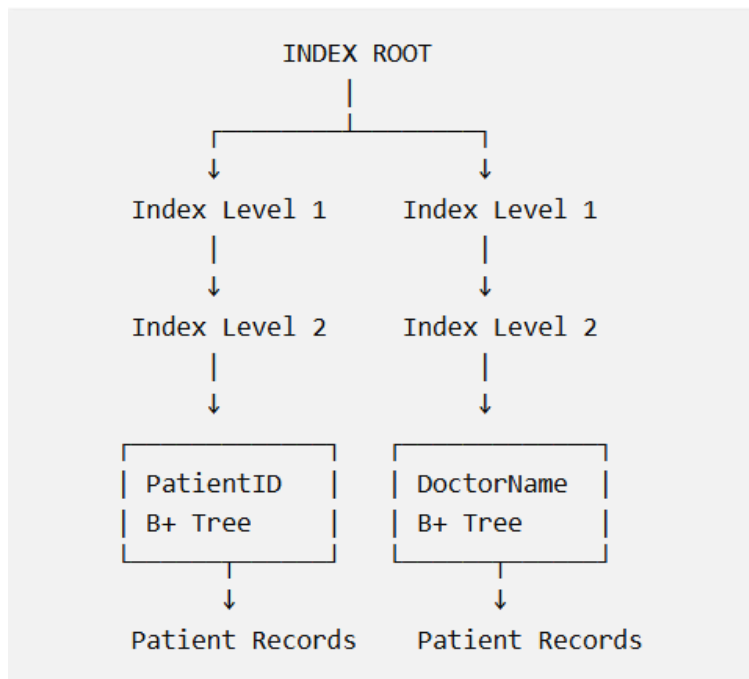
Example:

```
SELECT *
```

```
FROM Patient
```

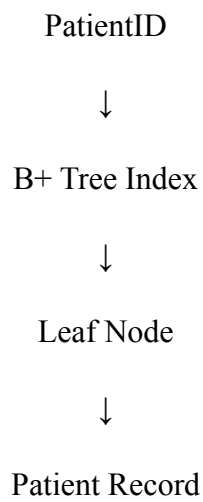
```
WHERE DoctorName = 'Dr. Kumar';
```

4. Multi-Level Index Structure



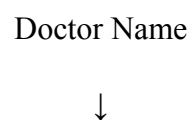
5. Working

For PatientID search:



This provides fast access to a specific patient.

For Doctor Name search:



Secondary B+ Tree



Matching Leaf Entries



Patient Records

This retrieves all patients belonging to the selected doctor.

6. Cost Analysis

For **N patient records**, a multi-level B+ Tree index provides search complexity approximately:

[
 $O(\log N)$
]

This is considerably more efficient than scanning every patient record.

Operation	Without Index	With Multi-Level Index
Search by PatientID	$O(N)$	$O(\log N)$
Search by DoctorName	$O(N)$	$O(\log N) + \text{matching records}$
Insert	Simple	Index update required
Delete	Simple	Index update required
Range/Multiple matching records	Slow	Efficient

Conclusion

A **multi-level B+ Tree indexing solution** with a primary index on **PatientID** and a secondary index on **DoctorName** is suitable for the healthcare system. It provides fast retrieval of individual patient records as well as efficient retrieval of all patients associated with a particular doctor.

Q4. Logistics Company – B-Tree vs B+ Tree

Question:

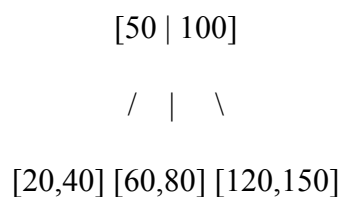
A logistics company needs to efficiently handle **dynamic insertions and deletions of shipment records**. Analyze whether **B-Tree or B+ Tree** is more suitable.

Answer

For a logistics company with frequent **insertions, deletions, and searches**, a **B+ Tree** is more suitable than a **B-Tree**.

1. B-Tree

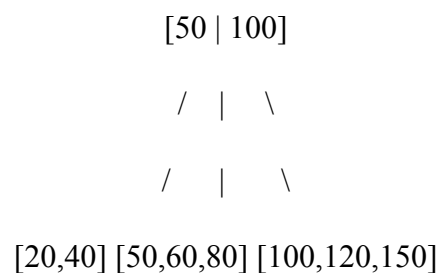
A B-Tree stores **keys and record pointers in both internal nodes and leaf nodes**.



Searching can sometimes finish at an internal node because data records can be stored at any level.

2. B+ Tree

A B+ Tree stores **index keys in internal nodes**, while the actual record pointers are stored at the **leaf level**. The leaf nodes are linked for sequential access.



3. Comparison

Feature	B-Tree	B+ Tree
Data storage	Internal + leaf nodes	Leaf nodes

Search	Fast	Very fast
Insertion	Efficient	Efficient
Deletion	Efficient	Efficient
Range queries	Good	Excellent
Sequential access	Less efficient	Highly efficient
Leaf nodes linked	Usually no	Yes
Database applications	Suitable	Highly suitable

4. Why B+ Tree is Better for Logistics

Shipment records are continuously **inserted, updated, and deleted** as shipments are created, delivered, cancelled, or modified.

A B+ Tree is preferable because:

- It maintains a balanced tree during insertions and deletions.
- It provides efficient searching even when the database becomes very large.
- It provides efficient sequential access to shipment records.
- Internal nodes allowing more keys per node and reducing tree height.
- Database systems commonly use B+ Tree structures for large indexed data.

5. Performance

For **N shipment records**, both structures provide approximately:

$$Search = O(\log N)$$

$$Insertion = O(\log N)$$

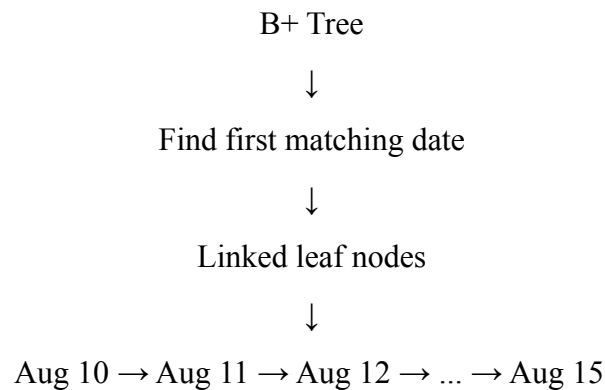
$$Deletion = O(\log N)$$

However, B+ Tree generally provides better performance for database workloads involving large datasets and range-based retrieval.

6. Example

Suppose the logistics company wants to find all shipments scheduled between **August 10 and August 15**.

With a B+ Tree:



The linked leaf nodes allow the system to scan the required range efficiently.

Conclusion

B+ Tree is more suitable for the logistics company because it efficiently supports **dynamic insertion, deletion, searching, and range queries**. Its linked leaf nodes and lower tree height make it particularly effective for large shipment databases.

Final Recommendation: B+ Tree.

Q5. HR System – Static Hashing vs. Extendible Hashing

Answer

For an HR system containing **5,000 employee records**, hashing can provide fast retrieval when searching for an employee using a unique attribute such as `Employee_ID`.

A suitable hashing scheme is to use **Employee_ID as the search key**.

1. Hashing Scheme

Assume the system uses:

$$h(\text{Employee_ID}) = \text{Employee_ID} \bmod 100$$

This creates **100 primary buckets** numbered from 0 to 99.

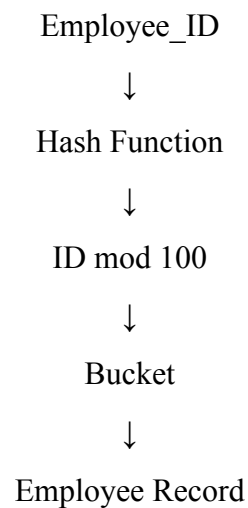
For example:

Employee ID = 1250

$$h(1250) = 1250 \bmod 100$$

$$= 50$$

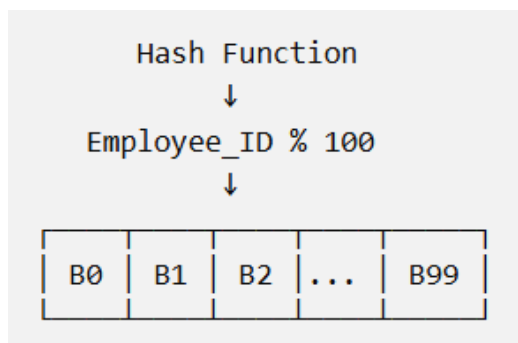
Therefore, employee 1250 is stored in **Bucket 50**.



2. Static Hashing

In **static hashing**, the number of buckets is fixed when the file is created.

For example:



Advantages

- Simple to implement.
- Fast direct access.
- Easy to maintain when the number of records is relatively stable.

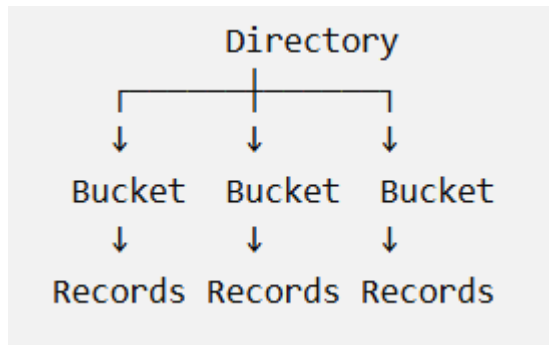
Disadvantages

- The number of buckets cannot automatically grow.
- As employees increase, **overflow buckets** may be required.

- Excessive collisions can reduce performance.

3. Extendible Hashing

Extendible hashing is a dynamic hashing technique in which the number of buckets can grow or shrink according to the number of records.



When a bucket becomes full, it can be split and the directory can be expanded.

Advantages

- Dynamically handles employee growth.
- Reduces long overflow chains.
- Efficient for insertion and deletion.
- Suitable when the database size changes frequently.

Disadvantages

- More complex than static hashing.
- Requires additional directory storage.
- Bucket splitting introduces some overhead.

4. Comparison

Feature	Static Hashing	Extendible Hashing
---------	----------------	--------------------

Number of buckets	Fixed	Dynamic
Implementation	Simple	More complex
Employee growth	Less suitable	Highly suitable
Overflow handling	Overflow buckets	Bucket splitting
Search	Fast	Fast
Insertions	Good for stable size	Good for changing size
Storage overhead	Low	Higher
Scalability	Limited	High

5. Evaluation for 5,000 Employees

If the HR system is expected to remain close to **5,000 employees**, static hashing with an appropriate number of buckets is simple and cost-effective.

However, if the company is expected to **hire many more employees**, extendible hashing is preferable because it can dynamically accommodate new records without requiring the entire hash structure to be redesigned.

Conclusion

For a fixed HR database of approximately **5,000 employees**, **static hashing** is a simple and efficient choice.

However, for a growing organization where employee records may increase significantly, **extendible hashing is the better long-term solution** because it dynamically expands the hash structure and handles overflow more efficiently.

Final Recommendation: Extendible Hashing for a growing HR system.

Q6. Airline Reservation System – Combined Index Structure

Answer

For an airline reservation system, two types of queries are required:

1. **Point query:** Search for a specific Flight_Number.
2. **Range query:** Retrieve flights within a particular **date range**.

A **combined B+ Tree indexing structure** using Flight_Number and Flight_Date is suitable.

1. Flight Table

Assume the table contains:

Attribute	Description
Flight_Number	Unique flight identifier
Flight_Date	Date of flight
Source	Departure location
Destination	Arrival location
Seats_Available	Available seats
Fare	Ticket fare

2. Index for Flight Number

Create a B+ Tree index on Flight_Number:

```
CREATE INDEX idx_flight_number  
ON Flights(Flight_Number);
```

This supports fast point queries.

Example:

```
SELECT *  
FROM Flights  
WHERE Flight_Number = 'AI202';
```

The index directly locates the required flight record.

3. Index for Flight Date

Create another B+ Tree index on Flight_Date:

```
CREATE INDEX idx_flight_date
```

```
ON Flights(Flight_Date);
```

This supports efficient range queries.

Example:

```
SELECT *
```

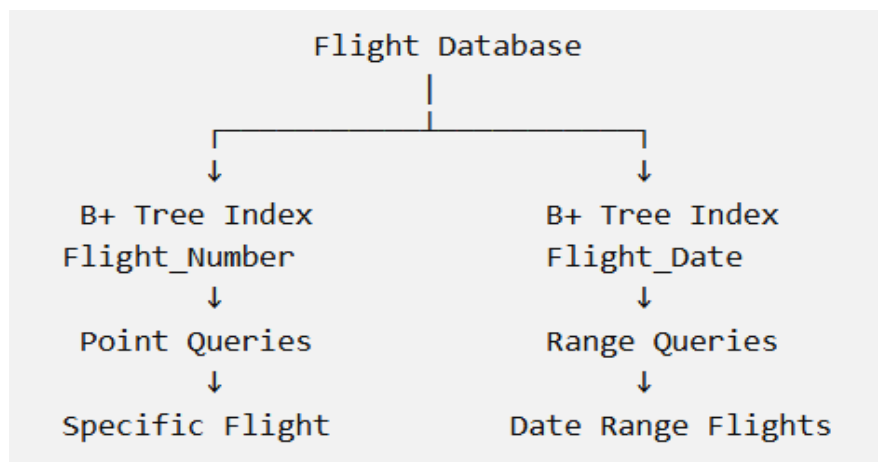
```
FROM Flights
```

```
WHERE Flight_Date
```

```
BETWEEN '2026-08-01' AND '2026-08-15';
```

The B+ Tree can locate the first matching date and then traverse the linked leaf nodes to retrieve the remaining dates.

4. Combined Index Structure



5. Composite Index

If the system frequently searches for flights using **both flight number and date**, a composite B+ Tree index can also be used:

```
CREATE INDEX idx_flight_date_combined
```

```
ON Flights(Flight_Number, Flight_Date);
```

For example:

```
SELECT *
FROM Flights
WHERE Flight_Number = 'AI202'
AND Flight_Date = '2026-08-10';
```

6. Performance Analysis

Query Type	Recommended Index	Performance
Search specific flight number	B+ Tree on Flight_Number	Fast
Search flights by date range	B+ Tree on Flight_Date	Very efficient
Search by flight number + date	Composite B+ Tree	Efficient
Sequential date retrieval	B+ Tree	Excellent

For **N records**, index traversal generally requires approximately:
 $O(\log N)$

For range queries, after finding the starting leaf, the linked leaf nodes allow the database to retrieve the required records efficiently.

Conclusion

The recommended solution is to maintain **two B+ Tree indexes**:

- **B+ Tree on Flight_Number** for fast point queries.
- **B+ Tree on Flight_Date** for efficient range queries.

A **composite index on (Flight_Number, Flight_Date)** can additionally be used when combined searches are frequent. This structure provides efficient and scalable retrieval for an airline reservation system.

Q7. University Student Database – File Organization and Secondary Indexing

Answer

For a university student database, the system needs to efficiently retrieve students based on:

1. **Department**
2. **CGPA range**

A suitable solution is to use **heap or sorted file organization with secondary B+ Tree indexes**.

1. Student Table

Assume the table contains:

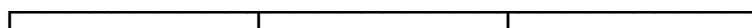
Attribute	Description
Student_ID	Unique student identifier
Student_Name	Student name
Department	Student's department
CGPA	Student's CGPA
Year	Academic year

2. File Organization

Use a **heap file organization** for storing student records.

In a heap file, records are stored in available disk locations without requiring a particular ordering.

Student Records



Block 1	Block 2	Block 3	
Student	Student	Student	
Records	Records	Records	

This is convenient for insertions because new student records can be placed in available space.

3. Secondary Index on Department

Create a **B+ Tree secondary index** on Department.

```
CREATE INDEX idx_department
```

```
ON Student(Department);
```

This allows fast retrieval of students belonging to a particular department.

Example:

```
SELECT *
```

```
FROM Student
```

```
WHERE Department = 'CSE';
```

The index directs the database to the relevant student records instead of scanning the complete table.

4. Secondary Index on CGPA

Create another B+ Tree index on CGPA.

```
CREATE INDEX idx_cgpa
```

```
ON Student(CGPA);
```

This is especially suitable for **range queries**.

Example:

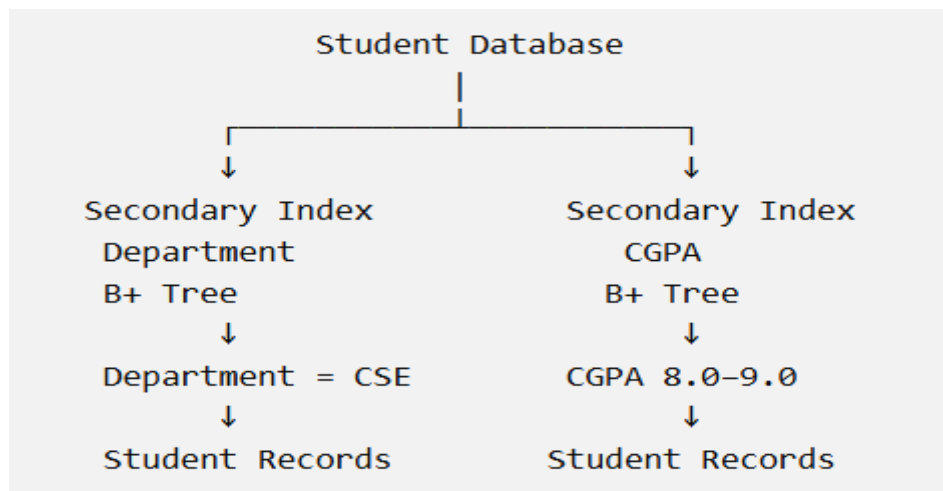
SELECT *

FROM Student

WHERE CGPA BETWEEN 8.0 AND 9.0;

The B+ Tree can locate the starting CGPA and traverse the linked leaf nodes to retrieve the required range.

5. Proposed Structure



6. Combined Query

The indexes can support queries such as:

SELECT *

FROM Student

WHERE Department = 'CSE'

AND CGPA BETWEEN 8.0 AND 9.0;

The database can use the department index and CGPA index to reduce the number of records that need to be examined.

7. Performance Analysis

Requirement	Technique	Performance
Insert student	Heap file	Fast

Search by department	B+ Tree secondary index	Fast
Search by CGPA range	B+ Tree secondary index	Very efficient
Department + CGPA query	Secondary indexes	Efficient
Large student database	Indexed storage	Scalable

B+ Tree searching generally takes approximately:
 $O(\log N)$

where **N** is the number of indexed records.

Conclusion

The recommended design is to use a **heap file organization** for storing student records and **secondary B+ Tree indexes on Department and CGPA**. The department index provides fast department-based searches, while the CGPA index efficiently handles range queries. This combination provides a practical and scalable solution for a university student database.

Q8. Disk Block Utilization for Heap, Sorted, and Hashed File Organizations

Given Data

- Number of records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

1. Records Per Block

$$\begin{aligned} \text{Records per block} &= \left\lfloor \frac{4096}{50} \right\rfloor \\ &= \lfloor 81.92 \rfloor = 81 \end{aligned}$$

Therefore, each block can store **81 records**.

2. Total Number of Blocks

$$\begin{aligned}\text{Number of blocks} &= \left\lceil \frac{100000}{81} \right\rceil \\ &= \boxed{1235 \text{ blocks}}\end{aligned}$$

3. Disk Space Used

$$1235 \times 4096 = 5,058,560 \text{ bytes}$$

Approximately:

$$\boxed{4.83 \text{ MB}}$$

The actual record data is:

$$100000 \times 50 = 5,000,000 \text{ bytes}$$

4. Block Utilization

The utilization is:

$$\begin{aligned}\text{Utilization} &= \frac{\text{Actual data}}{\text{Allocated disk space}} \times 100 \\ &= \frac{5,000,000}{5,058,560} \times 100 \\ &\approx \boxed{98.84\%}\end{aligned}$$

5.

The last block contains only the remaining:

$$100000 - (1234 \times 81) = 46$$

records.

So the last block contains:

$$46 \times 50 = 2300 \text{ bytes}$$

and has:

$$4096 - 2300 = 1796 \text{ bytes}$$

of unused space.

Comparison of File Organizations

File Organization	Block Requirement	Space Utilization	Main Advantage
-------------------	-------------------	-------------------	----------------

Heap File	1235 blocks	~98.84%	Fast insertion
Sorted File	1235 blocks	~98.84%	Efficient ordered/range access
Hashed File	Approximately 1235 data blocks*	Depends on bucket allocation	Fast equality search

*For hashed organization, actual utilization depends on the **number and size of hash buckets** and the collision/overflow strategy.

6. Analysis

Heap File Organization:

- Records are stored without a particular ordering.
- The 100,000 records require **1235 blocks**.
- It provides high storage utilization when blocks are filled efficiently.
- New records can be inserted easily.

Sorted File Organization:

- Records are maintained in sorted order.
- The same record size and block size give **1235 blocks** for the data.
- Storage utilization is approximately **98.84%**.
- Maintaining sorted order can increase insertion and deletion costs.

Hashed File Organization:

- Records are assigned to buckets using a hash function.
- It provides fast equality searches.
- Some space may be lost because of partially filled buckets or overflow blocks.
- Therefore, its practical utilization depends on the selected hash function and bucket design.

Conclusion

With **100,000 records of 50 bytes** and **4 KB blocks**, each block stores **81 records**, requiring **1235 blocks**. The resulting storage utilization is approximately **98.84%** for the data blocks. Heap and sorted organizations have the same basic block requirement, while hashed organization depends on its bucket and overflow design.

Q9. Social Media Platform – Composite Indexing Strategy

Answer

For a social media platform with **500 million posts**, a carefully designed **composite indexing strategy** is required to support fast retrieval by user, time, and hashtag.

1. Post Table

Assume the table contains:

Attribute	Description
post_id	Unique post identifier
user_id	User who created the post
timestamp	Date and time of the post
hashtag	Hashtag associated with the post
content	Post content

2. Primary Index

Create an index on post_id to uniquely identify each post.

```
CREATE UNIQUE INDEX idx_post_id
```

```
ON Posts(post_id);
```

This provides fast access to an individual post.

3. Composite Index on User and Timestamp

The most important composite index can be:

```
CREATE INDEX idx_user_timestamp
```

```
ON Posts(user_id, timestamp);
```

This index is useful for queries such as:

```
SELECT *
```

```
FROM Posts
```

```
WHERE user_id = 101
```

```
AND timestamp BETWEEN '2026-08-01' AND '2026-08-20';
```

The index first identifies the `user_id` and then efficiently searches the corresponding timestamp range.

4. Index on Hashtag

For hashtag-based searches, create a separate index:

```
CREATE INDEX idx_hashtag_timestamp
```

```
ON Posts(hashtag, timestamp);
```

This supports queries such as:

```
SELECT *
```

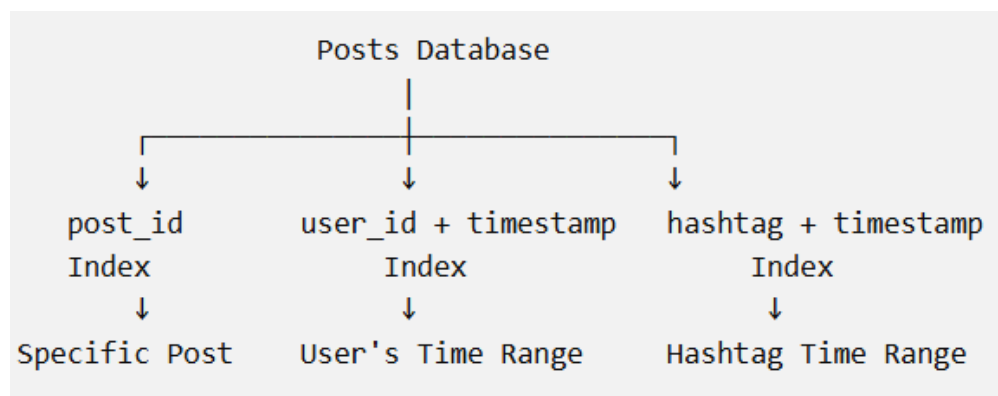
```
FROM Posts
```

```
WHERE hashtag = '#AI'
```

```
AND timestamp BETWEEN '2026-08-01' AND '2026-08-20';
```

The timestamp component also helps retrieve posts within a specific time period.

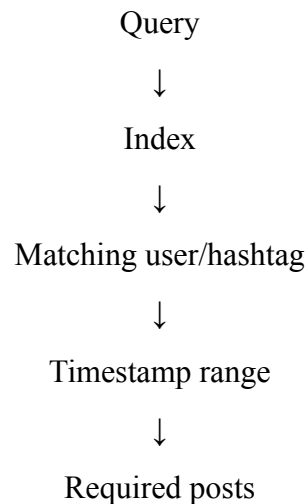
5. Proposed Composite Index Structure



6. Why Composite Indexing?

The platform contains **500 million posts**, so scanning the entire table for every query would be highly inefficient.

The composite indexes reduce the search space:



7. Index Selection

Query Requirement	Recommended Index
Find a specific post	post_id index
Find user's posts	user_id
Find user's posts by time	(user_id, timestamp)
Find posts with hashtag	hashtag
Find hashtag posts by time	(hashtag, timestamp)

8. Performance Analysis

B+ Tree indexes generally provide approximately:

$$O(\log N)$$

index traversal for searching, where **N = 500 million posts**.

The composite indexes are especially useful because they support both **equality filtering** and **timestamp range queries**.

However, indexes require additional storage and must be updated whenever posts are inserted or deleted. Therefore, only frequently used search attributes should be indexed.

Conclusion

For a social media platform containing **500 million posts**, the recommended strategy is:

- **B+ Tree index on post_id** for individual post retrieval.
- **Composite B+ Tree index on (user_id, timestamp)** for user-based and time-range queries.
- **Composite B+ Tree index on (hashtag, timestamp)** for hashtag and time-based queries.

This design provides fast and scalable retrieval while supporting the three important attributes: **user_id, timestamp, and hashtag**.

Q10. Supermarket Billing System – File Organization Comparison

Answer

A supermarket billing system generates a large number of transactions every day. Therefore, the file organization should provide efficient **insertion, deletion, and searching**.

1. Heap File

In a **Heap File**, records are stored in no particular order.

Operations:

- **Insert:** Very fast because the new transaction can be placed in available space.
- **Search:** Slow because records may need to be scanned sequentially.
- **Delete:** Relatively slow because the system must locate the record first.

Complexity:

$$Insert = O(1)$$

$$Search = O(N)$$

$$Delete = O(N)$$

2. Sorted File

In a **Sorted File**, transaction records are maintained in a particular order, such as transaction ID or date.

Operations:

- **Insert:** Slow because maintaining sorted order may require shifting records.
- **Search:** Fast using binary search or indexing.
- **Delete:** Can be costly because the sorted organization must be maintained.
- **Range queries:** Very efficient.

Complexity:

$$Search = O(\log N)$$

$$Insert \approx O(N)$$

$$Delete \approx O(N)$$

3. Hashed File

In a **Hashed File**, a hash function determines the storage location of each transaction.

For example:

$$h(Transaction_ID) = Transaction_ID \mod M$$

Operations:

- **Insert:** Fast when collisions are low.
- **Search:** Very fast for exact-match queries.
- **Delete:** Fast when the hash key is known.
- **Range queries:** Not efficient.

Average performance is approximately:

$$Search = O(1)$$

$$Insert = O(1)$$

$$Delete = O(1)$$

4. Comparison

Operation	Heap File	Sorted File	Hashed File
Insertion	Very Fast	Slow	Fast
Deletion	Slow	Slow	Fast
Exact Search	Slow	Fast	Very Fast
Range Search	Slow	Very Fast	Poor
Organization	Unordered	Sorted	Hash-based
Collision Handling	Not applicable	Not applicable	Required
Best Use	Frequent inserts	Range queries	Exact-match queries

5. Suitability for Supermarket Billing

For **1 million daily transactions**, the choice depends on the most frequent operation.

Heap File:

- Suitable when transactions are mainly inserted.
- Simple to maintain.
- Searching large numbers of transactions is inefficient.

Sorted File:

- Suitable when transactions are frequently retrieved by date or transaction ranges.
- Provides efficient range-based access.
- Frequent insertions can be expensive.

Hashed File:

- Best when the system frequently searches using an exact key such as Transaction_ID.
- Provides very fast average search and insertion.

- Not suitable for queries such as "all transactions between two dates."

6. Recommended Solution

For a supermarket billing system, **Hashed File Organization** is generally the best choice when the primary requirement is **fast transaction lookup using Transaction_ID**.

If the system frequently performs **date-range reports**, a **sorted/indexed structure** should additionally be considered.

Conclusion

For **1 million daily transactions**:

- **Heap File → Best for frequent insertion**
- **Sorted File → Best for ordered and range searches**
- **Hashed File → Best for fast exact-match searches**

Therefore, **Hashed File Organization** is recommended for the core billing transaction **lookup**, while indexing can be added for date-based reporting.